

# Pracownia programowania - Egzamin (pierwszy termin 2026) - Zestaw R27

Imię i nazwisko, numer albumu .....

Zasady egzaminu:

- **Zadanie 1: 6 pkt. Zadanie 2: 8 pkt. Zadanie 3: 10 pkt. Zadanie 4: 12 pkt. Zadanie 5: 14 pkt.**
- *Punktacja: 46-50 pkt - bdb(5,0); 41-45 pkt - db+(4,5); 36-40 pkt - db(4,0); 31-35 pkt - dst+(3,5); 26-30 pkt - dst(3,0); 0-25 pkt - ndst (2,0).*
- Obowiązuje regulamin zajęć.
- Czas: 90 minut.
- **Egzamin należy wykonać na komputerach zamontowanych na stałe w pracowniach.**
- Student przysyłając rozwiązania oświadcza, że rozwiązał je samodzielnie.
- W trakcie egzaminu nie można korzystać z żadnych materiałów pomocniczych w żadnej formie poza tablicą znaków ASCII udostępnioną jako pdf na pendrive. Wszelkie kody powinny być napisane manualnie bez korzystania z narzędzi automatycznie generujących kod (np. Copilot, chat GPT itp.).
- Publikowanie poleceń i rozwiązań w internecie jest zabronione do czasu napisania egzaminu przez wszystkie osoby.
- Kod musi się kompilować bez błędów, aby był sprawdzany. Ostrzeżenia (tzw. warningi) są dopuszczalne, o ile nie prowadzi to do błędów merytorycznych. Użycie innego kompilatora niż `gcc` może powodować brak niektórych konstrukcji.
- Kod zakomentowany nie będzie sprawdzany.
- W trakcie egzaminu zostanie udostępniony przez prowadzącego pendrive. Zawartość pendrive będzie zawierać pliki pomocnicze do poleceń. Ten sam pendrive służy do zgrania rozwiązań. Umieszczenie rozwiązań na pendrive powinno odbyć się w czasie egzaminu.
- Rozwiązania po czasie mogą nie być sprawdzane.
- W rozwiązaniach należy przestrzegać dobrych praktyk i konwencji nazw stosowanej na wykładzie. W przypadku gdy zaburzenie zaleceń spowoduje niejednoznaczność wykonania kodu (tzw. unexpected behavior), za dane polecenie mogą być obniżone punkty (nawet do zera). Zalecane jest jawne dołączenie używanych bibliotek poprzez `#include`.
- Używanie typu `bool` w rozwiązaniach jest zakazane.
- Po kartkach z poleceniami można pisać i traktować jako brudnopis.
- Wszystkie zadania mają być rozwiązane w postaci aplikacji konsolowych “jednoplikowych” (bez podziału na pliki nagłówkowe).
- Należy przestrzegać nazw funkcji i kolejności argumentów w poleceniach.
- Warto zwracać uwagę na typ zwracany funkcji — jeśli funkcja ma „coś” zwrócić, będzie to wskazane w poleceniu.
- W rozwiązaniach nie należy wykonywać nadmiarowych czynności, niewskazanych w poleceniu w ramach samodzielnie zdefiniowanej funkcji. Dodatkowe czynności mogą odbywać się w `main`. O ile w poleceniu nie zaznaczono inaczej, w rozwiązaniu można stworzyć funkcje pomocnicze.
- Format rozwiązania:
  - każde rozwiązanie w osobnym pliku z rozszerzeniem `.c` (mogą być w projekcie lub solo).
  - nazwy plików lub projektów: `zad1`, `zad2`, `zad3`, `zad4`, `zad5`
  - Zadania powinny być umieszczone w archiwum `.zip` na udostępnionym pendrive.
  - Nazwa archiwum powinna być wg schematu `NUMERZESTAWU_NUMERALBUMU.zip` gdzie numer zestawu znajduje się na górze kartki z poleceniami. np. `A23_123456.zip`
- Odstępstwa od zasad są możliwe jedynie po uzyskaniu wcześniejszej zgody na tzw. alternatywne sposoby zaliczeń.

Zad.1. Napisz funkcję `isPowerOfTwo`, której argumentem jest dodatnia liczba całkowita  $n$ . Funkcja sprawdza, czy liczba jest potęgą dwójki (czyli czy istnieje nieujemna liczba całkowita  $k$  taka, że  $2^k = n$ ). Funkcja zwraca 1 jeśli liczba jest potęgą dwójki oraz 0 w przeciwnym wypadku. Stwórz przypadek testowy dla funkcji.

Przykłady: 8 jest potęgą dwójki ( $2^3 = 8$ , zwraca 1), 1 jest potęgą dwójki ( $2^0 = 1$ , zwraca 1), natomiast 12 nie jest potęgą dwójki (zwraca 0).

Zad.2. Napisz funkcję `countUpper`, której argumentem jest tablica znaków. Funkcja ma zwrócić liczbę wielkich liter alfabetu angielskiego (A-Z) występujących w napisie. W zadaniu nie korzystaj z funkcji bibliotecznych poza instrukcjami wejścia/wyjścia. Stwórz przypadek testowy.

Przykłady:

- "Ala Ma Kota" → zwraca 3
- "PROGRAM" → zwraca 7
- "abc123" → zwraca 0

Zad.3. Napisz funkcję `countAbove`, której argumentem jest dwuwymiarowa tablica tablic o elementach typu `int`, jej wymiary  $n$  i  $m$  ( $n > 0, m > 0$ ) oraz liczba całkowita `value`. Funkcja ma zwrócić liczbę elementów tablicy, które są większe niż `value`. Stwórz przypadek testowy.

Przykład: dla tablicy postaci

$$\begin{bmatrix} 2 & 6 & 1 \\ 8 & -4 & 5 \end{bmatrix}$$

oraz `value = 4` funkcja ma zwrócić 3 (elementy większe od 4 to 6, 8 oraz 5; element -4 nie jest liczony).

Wskazówka: Tablica tablic powinna być zadeklarowana jako podwójny wskaźnik.

Zad.4. Napisz strukturę `Tank` (pol. zbiornik) z polami `label` (tablica znaków długości 20, pol. etykieta) oraz `liters` (typu `float`, pol. litry). Następnie napisz dwie funkcje i wywołaj każdą z nich co najmniej jeden raz:

- a) `newTank` - funkcja przyjmuje dwa argumenty: etykietę oraz początkową liczbę litrów, i zwraca wskaźnik na nowo utworzoną strukturę, ustawiając składowe z przekazanych argumentów. Dodatkowo funkcja powinna sprawdzić, czy początkowa liczba litrów jest nieujemna. W przeciwnym wypadku funkcja powinna zwracać `NULL`.
- b) `fill` - funkcja, której argumentami jest wskaźnik do struktury typu `Tank` oraz liczba litrów typu `float`. Funkcja ma dodać podaną liczbę litrów do zbiornika, ale tylko jeśli jest ona dodatnia.

Upewnij się, że drugą funkcję możesz wywołać w `main`.

Przykład: Dla zbiornika o etykiecie "A1" z liczbą litrów 50.0, po wywołaniu `fill` z wartością 20.0 liczba litrów powinna wynosić 70.0. Kolejne wywołanie `fill` z wartością -10.0 nie zmienia stanu zbiornika (wartość nie jest dodatnia).

Zad.5. Napisz funkcję `prefixSums`, która przyjmuje jako argument listę bez głowy o elementach typu:

```
struct elem {
    int x;
    struct elem * next;
};
```

Funkcja ma zamienić wartość każdego elementu listy na sumę prefiksową, czyli sumę tego elementu oraz wszystkich elementów go poprzedzających, modyfikując listę w miejscu. Funkcja nie zwraca wartości. Stwórz przypadek testowy.

Przykład: Dla listy 1, 2, 3, 4 po wykonaniu funkcji lista ma być postaci 1, 3, 6, 10 (kolejno: 1,  $1+2$ ,  $1+2+3$ ,  $1+2+3+4$ ). Dla listy 3, -1, 4 wynikiem jest lista 3, 2, 6 (sumy prefiksowe z liczbami ujemnymi).

Wskazówka: lista bez głowy to konstrukcja, w której elementy znajdują się od samego początku listy. Jeśli lista bez głowy ma  $n$  elementów, to do jej stworzenia potrzeba  $n$  struktur. Pusta lista bez głowy to NULL. Lista w `main` powinna być przechowywana jako wskaźnik do struktury.

